

# A Plain-Language Overview of Angular Manifold Routing

## A New Approach to Efficient AI Computation

### 1. Why This Matters

Modern artificial intelligence systems are extremely powerful, but they are also extremely expensive to run. Large AI models require enormous amounts of computing hardware, energy, and memory. Most of this cost comes from the fact that traditional neural networks perform large amounts of computation even when much of it is unnecessary.

Researchers have been exploring ways to make AI models **sparser** and **more selective** so that they only use compute where it is actually needed. If this can be done effectively, the same model could run:

- faster
- on less hardware
- using less energy
- at lower cost

The work summarized here explores a new routing method that appears to naturally produce this kind of efficiency.

### 2. The Core Idea

The central discovery is surprisingly simple:

*When token embeddings are normalized and routed based only on their direction (not their magnitude), tokens naturally cluster into far fewer routing buckets than expected.*

In simpler terms:

Instead of treating every input independently, the system **organizes similar inputs into shared directions in space**, which allows many of them to reuse the same computational resources.

This effect appears to arise from geometry alone.

### 3. A Helpful Analogy

Imagine a globe of the Earth.

Instead of assigning every point on Earth its own separate system, we divide the globe into **angular regions**, like slices of an orange.

Each region handles everything that points roughly in that direction.

Even if we increase the number of possible regions, something interesting happens:

People tend to cluster in certain areas rather than spreading evenly across the planet.

The same thing happens with AI embeddings.

Even when many routing slots are available, the inputs **naturally concentrate into a much smaller number of active regions**.

## 4. The Key Observation

If we create  $K$  routing sectors, we might expect the model to use roughly  $K$  of them.

Instead, the system consistently uses only about:

$$E(K) \approx c \cdot K^{0.57}$$

Where:

- $K$  = number of routing sectors available
- $E(K)$  = number of sectors actually used

Because the exponent is less than 1, the growth is **sublinear**.

In practical terms:

| Available Buckets | Buckets Actually Used |
|-------------------|-----------------------|
| 100               | ~40                   |
| 1,000             | ~150                  |
| 5,000             | ~350                  |

This means **most routing buckets remain unused**.

The result is **natural computational sparsity**.

## 5. Why This Is Important

This behavior has several practical benefits.

### 5.1 Less Computation

Only the active routing buckets need to run neural network layers.

### 5.2 Better Hardware Efficiency

When inputs concentrate into fewer buckets, memory access patterns become more predictable.

### 5.3 Improved Cache Performance

Our experiments showed **2.5× fewer cache misses** in some scenarios.

## 5.4 Lower Overall Cost

Across several tests, the system required **3–5× less effective routing work** compared to randomized routing.

## 6. What Causes This Effect

The key mechanism appears to be **angular geometry**.

The steps are:

1. Convert each token into a vector (embedding).
2. Normalize the vector so it lies on a **unit hypersphere**.
3. Ignore magnitude and consider only **direction**.
4. Divide the sphere into **angular sectors**.
5. Route tokens based on their direction.

Because embeddings often represent related concepts, they tend to occupy **similar directions in space**.

This causes tokens to **cluster naturally into sectors**.

## 7. What We Expected vs. What We Found

Originally, we expected routing to scale roughly like this:

$$E(K) = K$$

Meaning the system would use all buckets.

Instead, we consistently observed:

$$E(K) \approx K^{0.57}$$

This implies a **square-root-like scaling effect**, which is much more efficient.

## 8. What We Tested

A series of experiments were run to verify the phenomenon.

These included tests for:

- random routing vs geometric routing
- different normalization methods (L1, L2, L3, L4)
- routing capacity from  $K=25$  up to  $K=1000$
- hardware proxy models measuring memory traffic
- matched-progress training comparisons

Across all of these tests, the pattern remained stable.

The exponent varied slightly during training (between **0.50 and 0.64**) but remained clearly **sublinear**.

## 9. What Did Not Matter

Some factors turned out to have little or no effect.

### 9.1 Vector Norm Choice

Using different normalization types did not significantly change the scaling behavior.

### 9.2 Radial Shells

Adding radial partitions (distance from center) did not improve routing and sometimes made it worse.

This means the effect appears to be **purely angular**.

## 10. What This Means for AI Architecture

The results suggest a new type of routing architecture built around **angular manifolds**.

Instead of traditional gating networks deciding where computation happens, the geometry of the embedding space naturally organizes computation.

The pipeline becomes:

1. Embed tokens
2. Normalize to unit sphere
3. Partition directions into sectors
4. Route tokens by direction
5. Run compute only on active sectors
6. Aggregate outputs

This produces **automatic sparsity without complex gating mechanisms**.

## 11. Potential Benefits

If the effect holds in full-scale models, it could lead to:

- cheaper large AI systems
- faster inference
- lower energy consumption
- improved scaling for large models
- more efficient mixture-of-experts architectures

In other words, the same model capability could potentially run on **significantly less hardware**.

## 12. Current Status

So far this work has demonstrated:

- a reproducible routing compression effect
- strong scaling evidence
- hardware proxy benefits
- robustness across multiple normalization methods

However, the architecture still needs to be tested in **full production-scale models**.

That is the next step.

## 13. The Big Picture

The most interesting part of this discovery is that the efficiency appears to come from **geometry itself**.

The system does not need to learn complicated routing policies.

Instead, the structure of the embedding space naturally organizes computation.

This suggests that **geometric structure in representation space may be a powerful tool for building more efficient AI systems**.

## 14. In One Sentence

This work shows that when AI tokens are routed based on their direction on a normalized hypersphere, they naturally cluster into far fewer compute groups than expected, producing a scalable form of sparse computation that could significantly reduce hardware cost for large models.

## 15. What Comes Next

Future research will focus on:

- integrating this router into real transformer models
- testing performance on large training runs
- evaluating behavior at very large routing capacities
- studying how embeddings evolve during training

These experiments will determine whether the observed geometric scaling law can translate into real-world AI systems.

## Summary

The main finding is simple but powerful:

*Routing tokens by direction on a normalized hypersphere causes them to cluster into far fewer compute buckets than expected.*

This produces **automatic sparsity and improved hardware efficiency**, potentially enabling more scalable AI architectures.